

AUTONOMOUS AGENTS · SERIES TWO

The Scholar Series

Lesson 2 — Self-Planning.

A loop without a plan wanders. Teach the agent to draw its own map first.

Still building in LangGraph and CrewAI side by side

Where we are. In Lesson 1 our agent learned to run autonomously in a think-act-observe loop. But it improvised every step with no overall roadmap, so it wandered into tangents and sometimes covered the same ground twice. This lesson gives it the ability to *plan its own work* — to write an outline before it researches, and then follow that plan deliberately, in both frameworks.

What this lesson covers

- 1 **The scene** — watching the loop wander without a plan
- 2 **The concept** — plan-then-execute, and why the agent makes its own plan
- 3 **The architecture** — a planning phase added before the loop
- 4 **Build it in LangGraph** — a planner node feeding a list to the loop
- 5 **Build it in CrewAI** — a planning task feeding an execution task
- 6 **Compare** — how each framework expresses "plan, then do"
- 7 **Run it + the new crack** — and why a plan still produces an unchecked draft

PART 1

The scene: a capable wanderer

Lesson 1 gave us a real achievement: an agent that works autonomously, taking step after self-driven step. But if you watch its steps carefully, a frustrating pattern emerges. Here is its actual path on the battery briefing, laid bare.

Goal: Brief me on the current state of solid-state batteries.

step 1 → looked up the chemistry

step 2 → looked up leading companies

step 3 → looked up the chemistry *again*, from a slightly different angle

step 4 → wandered into a tangent on electric vehicle sales figures

The agent never covered the manufacturing hurdles or the timeline, which any good briefing needs, yet it spent two of its four steps on overlapping chemistry and a tangent. Why? Because in Lesson 1 it decided each next step *in the moment*, with no view of the whole. It's like a researcher with no outline, following whatever thread seems interesting next, with no guarantee of covering everything or avoiding repetition. The loop works, but it's aimless.

THE FAILURE, PRECISELY

Deciding each step in isolation, with no overall plan, means the agent can repeat itself, chase tangents, and leave important parts of the goal untouched. It has motion but no direction. What's missing is a *plan* made before the work begins — a map of what the finished briefing must contain.

PART 2

The concept: plan, then execute

The fix is a pattern called plan-then-execute, and it mirrors exactly how a thoughtful person tackles a big task. Before doing any research, you first sketch an outline: "to brief someone on solid-state batteries, I need to cover the technology, the leading players, the manufacturing hurdles, and the timeline." Only then do you go and research each point. The plan comes first and gives the work direction; the execution follows the plan.

The subtle but important part, and what makes this an *agentic* idea rather than just good advice, is that *the agent writes its own plan*. We don't hand it the outline. We give it the goal, and its first act is to

decompose that goal into a structured plan of its own making. Then it works through that plan point by point. This is a meaningful step up in autonomy from Lesson 1: the agent is no longer just choosing the next step reactively, it is looking at the whole goal and architecting an approach before it begins.

Contrast the two mental models directly. The Lesson 1 agent was like someone who starts walking and decides each turn as they reach it — they move, but they may circle back or miss streets. The Lesson 2 agent is like someone who studies the map first, plots a route that hits every destination, and then walks it. Same energy, far less waste, and a guarantee of covering everything that matters. The plan is what converts motion into progress.

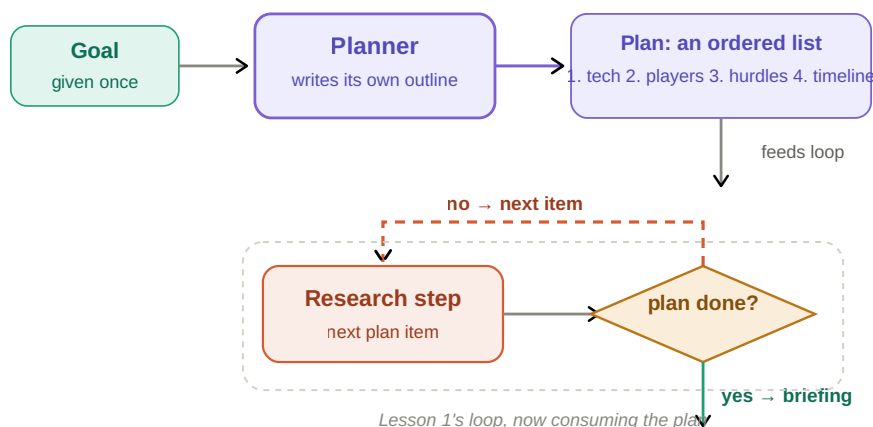
THE ONE IDEA OF THIS LESSON

Plan-then-execute means the agent first decomposes the goal into its own structured plan, then works through that plan deliberately rather than improvising each step. Crucially, the agent authors the plan itself — given only the goal, its first act is to architect the approach. Planning is what turns an aimless loop into directed work.

PART 3

The architecture: a planning phase before the loop

The change to our architecture is clean and intuitive. We keep Lesson 1's loop entirely, but we place a new *planning phase* in front of it. The agent first produces a plan — an ordered list of sub-topics to research — and then the loop works through that list one item at a time, instead of inventing each step from scratch.



Lesson 1's loop is intact on the right. The new piece is the planner on the left, which runs once at the start and hands the loop an ordered list to work through, giving every step a purpose.

Notice the loop now consumes the plan rather than improvising. Each pass through the loop picks up the next unaddressed item from the plan, researches it, and marks it done. The agent still loops autonomously, but now every iteration is anchored to a deliberate roadmap. This is the same accumulation pattern as series one: we kept what worked and added one new capability in front of it.

PART 4

Build it in LangGraph: a planner node feeds the loop

In LangGraph, the plan-then-execute shape is wonderfully literal: we add a `planner` node before the research loop, and the plan lives in the shared state where every later step can read it. Because LangGraph makes everything explicit, you can see the plan flowing from the planner into the loop.

```
from typing import TypedDict, List
from langgraph.graph import StateGraph, END
from langchain_openai import ChatOpenAI
from langchain_core.messages import SystemMessage, HumanMessage

llm = ChatOpenAI(model="gpt-4o-mini", temperature=0)

class ResearchState(TypedDict):
    goal: str
    plan: List[str]          # NEW: the agent's self-authored outline
    findings: List[str]
    current: int             # which plan item we're on now
```

Why this exists: the state gains a `plan` list and a `current` pointer. Together they replace Lesson 1's blind `steps` counter with a deliberate march through a roadmap.

```
def planner(state: ResearchState) -> ResearchState:
    # THE AGENT WRITES ITS OWN PLAN. Given only the goal, it decides
    # the sub-topics a complete briefing must cover.
    prompt = [
        SystemMessage(content="Break the research goal into 3-5 sub-topics "
                               "a complete briefing must cover. One per line."),
        HumanMessage(content=state["goal"]),
    ]
    plan_text = llm.invoke(prompt).content
    state["plan"] = [line.strip() for line in plan_text.splitlines() if line.strip()]
    state["current"] = 0
    return state
```

Why this exists: this node is the entire new idea. It runs once, before any research, and produces the roadmap. Notice we give it only the goal — the agent authors the plan itself.

```
def research_step(state: ResearchState) -> ResearchState:
    # Research the CURRENT plan item — not a freely-chosen topic.
```

```

    topic = state["plan"][state["current"]]
    finding = llm.invoke([SystemMessage(content="Research this sub-topic concisely."),
                        HumanMessage(content=topic)]).content
    state["findings"].append(f"{topic}: {finding}")
    state["current"] += 1      # advance to the next plan item
    return state

def plan_remaining(state: ResearchState) -> str:
    # Loop until every plan item is done – the plan IS the exit condition.
    return "continue" if state["current"] < len(state["plan"]) else "stop"

graph = StateGraph(ResearchState)
graph.add_node("planner", planner)
graph.add_node("research", research_step)
graph.set_entry_point("planner")      # plan FIRST
graph.add_edge("planner", "research") # then enter the loop
graph.add_conditional_edges("research", plan_remaining,
                           {"continue": "research", "stop": END})

agent = graph.compile()

```

Why this exists: the entry point is now **planner**, not the loop. The plan's length is the exit condition — far better than Lesson 1's arbitrary "stop after 4." The agent stops when the plan is complete.

PART 5

Build it in CrewAI: a planning task feeds an execution task

CrewAI expresses plan-then-execute in its own characteristic way: not as nodes in a graph, but as two tasks in sequence, where the output of the planning task flows into the execution task. CrewAI's **Process.sequential** runs them in order and passes results along automatically — the planning happens because you describe a planning task, not because you wire a node.

```

from crewai import Agent, Task, Crew, Process

planner = Agent(
    role="Research Planner",
    goal="Decompose a topic into the sub-topics a briefing must cover",
    backstory="You are an editor who maps out coverage before any "
            "writing begins, ensuring nothing important is missed.",
)

analyst = Agent(
    role="Research Analyst",
    goal="Research each planned sub-topic thoroughly",
    backstory="You work methodically through a given outline.",
)

```

Why this exists: we now have two agents with distinct roles — one that plans, one that executes. CrewAI's role metaphor makes "the planner" and "the analyst" feel like two members of a small team.

```
# The planning task produces the outline...
plan_task = Task(
    description="Create an outline of sub-topics for: solid-state batteries.",
    expected_output="A numbered list of 3-5 sub-topics to research.",
    agent=planner,
)

# ...and the research task CONSUMES it. context= passes the plan along.
research_task = Task(
    description="Research each sub-topic from the outline and write the briefing.",
    expected_output="A complete briefing covering every outlined sub-topic.",
    agent=analyst,
    context=[plan_task], # the plan flows in as context
)

# Sequential process: plan first, then execute, in order.
crew = Crew(agents=[planner, analyst], tasks=[plan_task, research_task],
            process=Process.sequential)
result = crew.kickoff()
```

Why this exists: `context=[plan_task]` is how CrewAI passes the plan from one task to the next.

`Process.sequential` guarantees the plan is made before the research runs — plan-then-execute, the CrewAI way.

PART 6

Compare: two ways to say "plan first"

Both frameworks achieve plan-then-execute, but the way they express it reveals their differing philosophies again, and the contrast is sharper here than in Lesson 1.

LANGGRAPH VS CREWAI — PLANNING

	LangGraph	CrewAI
Plan lives in	The shared state, as a list you can inspect	A task's output, passed via context
Ordering	You set the entry point and edges	Process.sequential handles order for you
Visibility	You can read and modify the plan mid-run	The plan flows automatically; less to touch
Feels like	Engineering a pipeline	Briefing a small team
Best when	You need to inspect or alter the plan in code	You want clean role separation with little wiring

The deeper lesson here is about *where the plan lives*, and it's a genuinely useful distinction to carry forward. In LangGraph the plan is data in your state — you can print it, edit it, branch on it, because it's an explicit object you own. In CrewAI the plan is the output of one task handed to the next as context — you describe the hand-off and the framework manages it. If your application needs to examine or adjust the plan partway through, LangGraph's explicit state is a real advantage. If you simply want a planner and an executor to cooperate with minimal plumbing, CrewAI's task hand-off is more elegant. Same pattern, different centre of gravity — and knowing which you need is the judgement this series is training.

PART 7

Run it, and the new crack

Run either version and the wandering disappears. The agent now plans, then marches through that plan with purpose.

Goal: Brief me on the current state of solid-state batteries.

[plan] 1. core technology 2. leading companies 3. manufacturing hurdles 4. timeline

[execute] researched item 1 → 2 → 3 → 4, each once, in order

Scholar: A briefing that systematically covers the technology, the key players, the manufacturing hurdles, and a grounded timeline — every part the goal demanded, nothing repeated.

The contrast with Lesson 1 is striking. No tangents, no repeated chemistry, and crucially, every part a good briefing needs is covered, because the plan guaranteed it. The agent converted the same autonomous energy into directed, complete work. But now look critically at what it produced — and here's the subtle new problem. The agent followed its plan and wrote a draft, then immediately declared itself finished. It never *read its own draft* to ask whether it was actually any good. What if a section is thin? What if a claim lacks support? What if the conclusion doesn't follow from the findings? The agent has no idea, because it never looked back.

THE PROBLEM LESSON 3 WILL FIX

Our agent now plans and executes, but it accepts its own first draft without question — it never critiques its work. In Lesson 3, the most powerful idea in this series, we'll give the agent the ability to *reflect*: to read its own output critically, identify weaknesses, and revise. The agent is about to become its own toughest editor.

End of Scholar Lesson 2. You taught the agent to author its own plan and execute it deliberately, in both frameworks, and learned where each keeps the plan. Next: teaching the agent to judge and improve its own work.